SQL QUICK REVISION

Section No.	Topic	Key Concepts Covered
01	SQL Introduction	What is SQL, Why SQL, DDL, DML, DQL, DCL, TCL
02	Databases and Tables	CREATE DATABASE, USE DATABASE, CREATE TABLE, Datatypes
03	Constraints	PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT
04	CRUD Operations	INSERT, SELECT, UPDATE, DELETE
05	Querying Data	SELECT, DISTINCT, ORDER BY, LIMIT, Aliases
06	Filtering Data	WHERE, AND, OR, NOT, BETWEEN, IN, NOT IN, LIKE
07	SQL Operators	Arithmetic, Comparison, Logical, Bitwise Operators
08	Aggregate Functions	COUNT(), SUM(), AVG(), MIN(), MAX()
09	Grouping Data	GROUP BY, HAVING, WHERE vs HAVING
10	Table Operations	ALTER TABLE, ADD COLUMN, MODIFY COLUMN, DROP COLUMN, TRUNCATE
11	Joins	INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN, SELF JOIN
12	Set Operations	UNION, UNION ALL, INTERSECT, EXCEPT/MINUS
13	Subqueries	Nested Queries, Correlated Subqueries, Scalar Subqueries
14	Views	CREATE VIEW, Updating Views, Advantages & Limitations
15	Window Functions	ROW_NUMBER(), RANK(), DENSE_RANK(), LEAD(), LAG()
16	Common Table Expressions (CTEs)	Basic CTEs, Multiple CTEs, Recursive CTEs
17	CASE Statements	Conditional Logic Using CASE
18	Indexing	Indexes, Clustered vs Non-Clustered Indexes
19	Triggers	BEFORE Trigger, AFTER Trigger, Trigger Use Cases
20	Stored Procedures	Procedures, Parameters, Reusable SQL Logic
21	Transactions	COMMIT, ROLLBACK, SAVEPOINT

01. SQL INTRODUCTION

What is SQL?

SQL (Structured Query Language) is a language used to store, retrieve, update, and manage data in relational databases..

TYPES OF SQL COMMANDS

Category	Full Form	Purpose	Common Commands
DDL	Data Definition Language	Used to define and modify database structures.	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Used to insert, update, and delete data in tables.	INSERT, UPDATE, DELETE
DQL	Data Query Language	Used to retrieve data from databases.	SELECT
DCL	Data Control Language	Used to manage user permissions and access control.	GRANT, REVOKE
TCL	Transaction Control Language	Used to manage database transactions and ensure data consistency.	COMMIT, ROLLBACK, SAVEPOINT

02. DATABASES & TABLES

CREATE DATABASE

Used to create a new database.

Syntax:

CREATE DATABASE database_name;

USE DATABASE

Used to select and work with a specific database.

Syntax:

USE database_name;

DROP DATABASE

Used to permanently delete a database along with all its tables and data.

Syntax:

DROP DATABASE database_name;

DATATYPES

DATATYPE	DESCRIPTION	EXAMPLE
CHAR(N)	Stores fixed-length character data.	CHAR(10)
VARCHAR(N)	Stores variable-length character data.	VARCHAR(50)
INT	Stores integer values.	100
TINYINT	Stores small integer values.	127
DOUBLE	Stores decimal/floating-point values.	99.99
BOOLEAN	Stores TRUE or FALSE values.	TRUE
DATE	Stores dates in YYYY-MM-DD format.	2026-06-13
YEAR	Stores year values.	2026
BLOB	Stores Binary Large Objects such as images, audio, or files.	Binary Data
SIGNED	Allows both positive and negative values.	-100, 100
UNSIGNED	Allows only positive values.	0, 100

03. CONSTRAINTS

Constraints are rules applied to table columns to ensure data accuracy and integrity.

Constraint	Description	Example Use Case
NOT NULL	Prevents a column from storing NULL values.	Email, Aadhaar Number
UNIQUE	Ensures all values in a column are unique.	Aadhaar Number, Username
PRIMARY KEY	Uniquely identifies each record in a table. Cannot contain NULL values.	Student_ID, Employee_ID
FOREIGN KEY	Creates a relationship by referencing the Primary Key of another table.	Course_ID referencing Course table
DEFAULT	Assigns a default value if no value is provided by the user.	Default Age = 18
CHECK	Validates data before insertion based on a condition.	Age > 18, Aadhaar length = 12

Example

```
CREATE TABLE Student(
  student_id  INT  PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
```

```
  email VARCHAR(100) UNIQUE,
  age INT CHECK(age >= 18),
  city VARCHAR(20) DEFAULT 'Pune'
);
```

04. CRUD OPERATIONS

CRUD represents the four basic operations performed on data in a database.

Operation	Description
INSERT	Used to insert new data into a table.
SELECT	Used to retrieve data from a table. SELECT * retrieves all columns.
UPDATE	Used to modify existing records in a table.
DELETE	Used to remove records from a table.

05. QUERYING DATA

Clause	Description
SELECT	Used to retrieve data from a table.
DISTINCT	Returns only unique values.
LIMIT	Restricts the number of rows returned.
ORDER BY	Sorts data in ascending or descending order.
Aliases (AS)	Provides a temporary short name for a column or table.

06. FILTERING DATA

Clause	Description
WHERE	Filters records based on a condition.
AND	Returns records when all conditions are true.
OR	Returns records when at least one condition is true.
NOT	Reverses a condition.
BETWEEN	Filters values within a specified range.
IN	Checks whether a value exists in a given list.
NOT IN	Checks whether a value does not exist in a given list.
LIKE	Used for pattern matching.

Wildcards	Special characters (%, _) used with LIKE.

07. SQL OPERATORS

Operator Type	Description
Arithmetic Operators	Perform mathematical operations (+, -, *, /, %).
Comparison Operators	Compare two values (=, >, <, >=, <=, !=).
Logical Operators	Combine multiple conditions (AND, OR, NOT).
Bitwise Operators	Perform operations on binary values (&, `

08. AGGREGATE FUNCTIONS

Aggregate functions perform calculations on a group of rows and return a single value.

Function	Description
COUNT()	Returns the number of records.
SUM()	Returns the total sum of values.
AVG()	Returns the average value.
MIN()	Returns the smallest value.
MAX()	Returns the largest value.

09. GROUPING DATA

Clause	Description
GROUP BY	Groups rows having the same values in specified columns.
HAVING	Filters grouped data based on a condition.
WHERE vs HAVING	WHERE filters rows before grouping, while HAVING filters groups after grouping.

10.TABLE OPEARTIONS

Command	Description	Syntax
ALTER TABLE	Used to modify the structure of an existing table.	ALTER TABLE table_name;
ADD COLUMN	Adds a new column to a table.	ALTER TABLE table_name ADD column_name datatype;
MODIFY COLUMN	Changes the datatype or properties of an existing column.	ALTER TABLE table_name MODIFY column_name datatype;
CHANGE COLUMN	Renames a column and/or changes its datatype.	ALTER TABLE table_name CHANGE old_column_name new_column_name datatype;
DROP COLUMN	Removes a column from a table.	ALTER TABLE table_name DROP COLUMN column_name;
RENAME TABLE	Renames an existing table.	ALTER TABLE old_table_name RENAME TO new_table_name;
TRUNCATE TABLE	Removes all records but keeps the table structure.	TRUNCATE TABLE table_name;

11. JOINS ⭐

Why Joins?

Joins are used to retrieve and combine data from two or more related tables.

Join Type	Description
INNER JOIN	Returns only the matching records present in both tables.
LEFT JOIN	Returns all records from the left table and matching records from the right table.
RIGHT JOIN	Returns all records from the right table and matching records from the left table.
FULL JOIN	Returns all records from both tables, including matching and non-matching records.
CROSS JOIN	Returns the Cartesian product of both tables (every row from the first table combined with every row from the second table).
SELF JOIN	Joins a table with itself to compare or relate rows within the same table.

12. SET OPERATIONS

Set Operation	Description
UNION	Combines results from multiple queries and removes duplicate records.
UNION ALL	Combines results from multiple queries and keeps duplicate records.
INTERSECT	Returns only the common records present in both query results.
EXCEPT / MINUS	Returns records from the first query that are not present in the second query.

13. SUBQUERIES ⭐

A Subquery is a query inside another query.

- Single Row Subquery → Returns one row.
- Multiple Row Subquery → Returns multiple rows.
- Correlated Subquery → Depends on the outer query for execution.
- Nested Query → Query written inside another query, possibly with multiple levels.

14. VIEWS

A View is a virtual table created from a SQL query.

- CREATE VIEW → Creates a virtual table.
- Updating Views → Changes made in the view affect the original table (if the view is updatable).
- Advantages → Security, simplicity, reusability.
- Limitations → Some views are not updatable and may affect performance.

15. WINDOW FUNCTIONS

Function	Description
OVER()	Defines the window (set of rows) on which a window function operates.
ROW_NUMBER()	Assigns a unique sequential number to each row within a partition.
RANK()	Assigns ranks to rows; duplicate values receive the same rank and gaps are created.
DENSE_RANK()	Assigns ranks to rows; duplicate values receive the same rank but no gaps are created.
LEAD()	Retrieves data from the next row without using a self join.
LAG()	Retrieves data from the previous row without using a self join.

16. COMMON TABLE EXPRESSIONS (CTES) ⭐

A CTE is a temporary result set used to simplify complex queries.

- **Basic CTE** → Temporary result set used in a query.
- **Multiple CTEs** → Multiple temporary result sets in one query.
- **Recursive CTE** → References itself; used for hierarchical data.

17. CASE STATEMENTS

Used to apply conditional logic in SQL (similar to IF-ELSE).

- Returns different values based on conditions.
- Commonly used in SELECT, UPDATE, and ORDER BY.

18. INDEXING

An Index improves query performance by speeding up data retrieval.

- Improves SELECT performance.
- Can be created on one or more columns.

Advantages: Faster searching and sorting.

Drawbacks: Extra storage and slower INSERT, UPDATE, DELETE.

19. TRIGGERS

A Trigger automatically executes when an event occurs on a table.

- **BEFORE Trigger** → Executes before INSERT, UPDATE, or DELETE.
- **AFTER Trigger** → Executes after INSERT, UPDATE, or DELETE.

Uses: Validation, auditing, logging, automatic updates.

20. STORED PROCEDURES

A Stored Procedure is a reusable collection of SQL statements stored in the database.

- **Input Parameters** → Accept values from the user.
- **Output Parameters** → Return values to the caller.

Benefits: Reusability, better performance, easier maintenance.

21. TRANSACTIONS (TCL)

Used to maintain data consistency and integrity.

- **COMMIT** → Permanently saves changes.
- **ROLLBACK** → Undoes changes.
- **SAVEPOINT** → Creates a checkpoint for partial rollback.

Importance: Prevents partial updates and ensures reliable transactions.